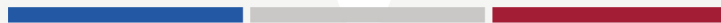




MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING



MODULE 1 UNIT 1
Lesson Video 2 Transcript

Module 1 Unit 1 Lesson Video 2 Transcript

PAUL MCDONAGH-SMITH: In the previous video, we positioned agentic AI on the AI continuum and established a critical distinction, the shift from reactive systems to proactive ones. Now I want to take you inside that shift, because understanding the architecture is what separates a leader who can evaluate an AI initiative from one who simply approves a budget line. So, how does a large language model become agentic?

The reactive baseline

PAUL MCDONAGH-SMITH: A large language model, an LLM, is at its core a next token prediction engine. It has been trained on vast amounts of data and can generate remarkably fluent, contextually appropriate responses. When you type a question, a prompt into ChatGPT or Claude or Gemini, the model reads your prompt and produces an output. One input, one output. The conversation might continue, but each exchange is essentially based on the same transactional trajectory. You prompt, it outputs. This is what we mean by reactive. The system has no goals of its own. It doesn't plan. It simply generates the most probable next token in a sequence of words, musical notes, video frames.

And here's the thing that is already extraordinarily useful. Reactive LLMs have transformed knowledge work. Drafting, summarizing, coding, translating, and they have a ceiling. They are, to borrow an analogy, like an expert witness in a courtroom. Brilliant when asked the right question, but not the one running the case.

What changes: Tools, reason and the loop

PAUL MCDONAGH-SMITH: An agent is what you get when you wrap that LLM in a system that gives it the ability to act, not just respond. And that system has several key components.

First, tool access. An agent can call external tools, a database query, a web search, an API, a code interpreter. This is fundamental. It means the system is no longer limited to what it has learned during training. It can go and get information. It can take action in the world. The agent communicates not only with the human, but with a suite of tools that extends its capabilities.

Second, planning and task decomposition. An agent can take a complex objective and break it into subtasks. If you ask a reactive LLM to prepare a competitive analysis, it will generate a single or multiple documents in one pass. An agentic system might instead decompose that into identify competitors, retrieve recent financial data, compare product positioning, draft the analysis, review for accuracy. Each step is a distinct action executed in sequence.

And then thirdly, memory and context management. Agents maintain state across a workflow. They remember what has been done, what remains, and what the constraints are. This is what allows them to handle multi-step processes without losing the thread. Something a stateless LLM conversation can't easily sustain.

And fourthly, and perhaps most interestingly, feedback and self-correction. An agent can evaluate its own output, identify errors, and revise. Research has shown that simply asking the model to think step by step, or to review and reflect on its answer, can improve accuracy without changing a single parameter in the model itself. You're not retraining the system, you're changing how it searches for the answer. This is what the field calls reasoning, and it's one of the most consequential developments in AI today.

So, here is the conceptual pivot. With a reactive LLM, you query, it retrieves. If you give an agentic system a brief, it will metaphorically go away, pull data, run calculations, draft a memo, check it against your style guide, and bring you something to review. It's not perfect, it will still need supervision, but it's not waiting for you to dictate every step.

That shift from respond to pursue is where a dimension of the organizational value lives. It's the difference between AI that assists a workflow and AI that executes one. But it's also where the risk lives, Because the moment a system can take action autonomously query a database, send an email, trigger a purchase order, the consequences of errors compound. A wrong answer in a chatbot, at best is an inconvenience, at worst deeply damaging to brand reputation. A wrong answer in a supply chain is a financial exposure. And so the question for leaders isn't simply can we make this agentic, but where does autonomy add value, and where does it add risk?

The relationship between an LLM and an agent isn't one of replacements, it's one of augmentation. The LLM provides the reasoning engine, the agent architecture, the tools, the planning, the memory, the guardrails, providing the scaffolding that makes that engine operationally useful.

And this distinction matters for your organization because the decision you face isn't should we use AI? The decision is for which workflows, at what level of autonomy, with what types of guardrails. That is the design problem, and it's a management and leadership problem, not a technology problem.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.